

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №1
СОА
Вариант 67205

Выполнил:

Вайнштейн Владислав Витальевич Р3418
Богоявленский Александр Викторович Р3417

Практик:

Цопа Евгений Алексеевич

Санкт-Петербург, 2026

Содержание

1. Цель работы	3
2. Ход работы	4
3. Ссылка на гит	5
4. Вывод	5

1. Цель работы

(СОА) Лабораторная работа №1

Введите вариант:

67205

Внимание! У разных вариантов разный текст задания!

Разработать спецификацию в формате OpenAPI для набора веб-сервисов, реализующего следующую функциональность:

Первый веб-сервис должен осуществлять управление коллекцией объектов. В коллекции необходимо хранить объекты класса `StudyGroup`, описание которого приведено ниже:

```
public class StudyGroup {
    private Integer id; //Поле не может быть null, Значение поля должно быть больше 0, Значение
    этого поля должно быть уникальным, Значение этого поля должно генерироваться автоматически
    private String name; //Поле не может быть null, Строка не может быть пустой
    private Coordinates coordinates; //Поле не может быть null
    private java.time.LocalDate creationDate; //Поле не может быть null, Значение этого поля
    должно генерироваться автоматически
    private Long studentsCount; //Значение поля должно быть больше 0, Поле не может быть null
    private long expelledStudents; //Значение поля должно быть больше 0
    private FormOfEducation formOfEducation; //Поле не может быть null
    private Semester semesterEnum; //Поле может быть null
    private Person groupAdmin; //Поле может быть null
}
```

```
public class Coordinates {
    private long x; //Максимальное значение поля: 726
    private Integer y; //Максимальное значение поля: 189, Поле не может быть null
}

public class Person {
    private String name; //Поле не может быть null, Строка не может быть пустой
    private Float weight; //Поле не может быть null, Значение поля должно быть больше 0
    private String passportID; //Поле может быть null
    private Location location; //Поле может быть null
}

public class Location {
    private long x;
    private long y;
    private String name; //Строка не может быть пустой, Поле может быть null
}

public enum FormOfEducation {
    DISTANCE_EDUCATION,
    FULL_TIME_EDUCATION,
    EVENING_CLASSES;
}

public enum Semester {
    FIRST,
    THIRD,
    SEVENTH;
}
```

Веб-сервис должен удовлетворять следующим требованиям:

- API, реализуемый сервисом, должен соответствовать рекомендациям подхода RESTful.
- Необходимо реализовать следующий базовый набор операций с объектами коллекции: добавление нового элемента, получение элемента по ИД, обновление элемента, удаление элемента, получение массива элементов.
- Операция, выполняемая над объектом коллекции, должна определяться методом HTTP-запроса.
- Операция получения массива элементов должна поддерживать возможность сортировки и фильтрации по любой комбинации полей класса, а также возможность分页ного вывода результатов выборки с указанием размера и порядкового номера выводимой страницы.
- Все параметры, необходимые для выполнения операции, должны передаваться в URL запроса.
- Информация об объектах коллекции должна передаваться в формате json.
- В случае передачи сервису данных, нарушающих заданные на уровне класса ограничения целостности, сервис должен возвращать код ответа http, соответствующий произошедшей ошибке.

Помимо базового набора, веб-сервис должен поддерживать следующие операции над объектами коллекции:

- Рассчитать сумму значений поля expelledStudents для всех объектов.
- Вернуть количество объектов, значение поля studentsCount которых равно заданному.
- Вернуть массив объектов, значение поля name которых начинается с заданной подстроки.

Эти операции должны размещаться на отдельных URL.

Второй веб-сервис должен располагаться на URL `/isu`, и реализовывать ряд дополнительных операций, связанных с вызовом API первого сервиса:

- `/group/{from-group}/move/{to-group}` : переместить всех студентов из одной группы в другую
- `/group/{group-id}/change-edu-form/{new-form}` : сменить формат обучения группы на указанный

Эти операции также должны размещаться на отдельных URL.

Для разработанной спецификации необходимо сгенерировать интерактивную веб-документацию с помощью Swagger UI. Документация должна содержать описание всех REST API обоих сервисов с текстовым описанием функциональности каждой операции. Созданную веб-документацию необходимо развернуть на сервере `helios`.

2. Ход работы

ITMO STUDENTS REST API 0.0.1 OAS 3.0

REST API для управления коллекцией учебных групп StudyGroup.

Спецификация описывает два веб-сервиса:

1. Основной сервис управления коллекцией StudyGroup.
2. Второй сервис /isu, выполняющий дополнительные операции посредством вызова API первого сервиса.

Study Groups CRUD Базовые операции управления коллекцией StudyGroup

GET	/study-groups	Получить массив учебных групп	⌵
POST	/study-groups	Добавить учебную группу	⌵
GET	/study-groups/{id}	Получить учебную группу по ID	⌵
PUT	/study-groups/{id}	Обновить учебную группу	⌵
DELETE	/study-groups/{id}	Удалить учебную группу	⌵

Study Groups Statistics Статистические операции над коллекцией StudyGroup

GET	/study-groups/expelled-students/sum	Рассчитать сумму expelledStudents	⌵
GET	/study-groups/students-count/{studentsCount}/count	Получить количество групп с заданным studentsCount	⌵
GET	/study-groups/name/starts-with/{prefix}	Найти группы по началу имени	⌵

POST `/isu/group/{from-group}/move/{to-group}` Переместить студентов из одной группы в другую

POST `/isu/group/{group-id}/change-edu-form/{new-form}` Изменить форму обучения группы

GET `/study-groups` Получить массив учебных групп

Возвращает массив объектов StudyGroup.

Поддерживаются фильтрация, сортировка и постраничный вывод результатов.

Parameters

Try it out

Name	Description
filter array<string> (query)	Условия фильтрации в LHS bracket-нотации <code>field[operator]-value</code> . Каждое условие передается отдельным параметром <code>filter</code> , параметры объединяются через <code>&</code> . Все условия объединяются по И. Пример: <code>GET /study-groups?filter=name[like]=Math&filter=studentsCount[gte]=50&filter=studentsCount[lte]=200</code> Поддерживаемые поля: id, name, coordinates x, coordinates y, createDate, studentsCount, expelledStudents, formOfEducation, semesterEnum, groupAdmin.name, groupAdmin.weight, groupAdmin.passportID, groupAdmin.location.x, groupAdmin.location.y, groupAdmin.location.name. Операторы: eq, ne, like, ilike, notlike, notilike, gt, lt, gte, lte, inlist, notinlist, isnull. Examples: Одно условие <code>name[like]=Math</code>
sort array<string> (query)	Условия сортировки в нотации <code>field[direction]</code> . Каждое поле сортировки передается отдельным параметром <code>sort</code> , параметры объединяются через <code>&</code> . Порядок применения определяется порядком следования параметров в строке запроса. Пример: <code>GET /study-groups?sort=name[asc]&sort=studentsCount[asc]</code> Список полей — тот же, что и у параметра <code>filter</code> . Направления: asc, desc. Examples: Сортировка по одному полю <code>studentsCount[desc]</code>
page	Порядковый номер страницы. Нумерация страниц начинается с 0.

3. Ссылка на гит

<https://github.com/bogoyavlensciiAV/SOA/>

4. Вывод

В ходе лабораторной работы разработана OpenAPI-спецификация для двух взаимодействующих RESTful-сервисов. Все ограничения целостности данных и соответствующие HTTP-статусы учтены, а спецификация преобразована в интерактивную документацию Swagger UI, развёрнутую на сервере helios.